Evaluación del rendimiento del procesador vectorial VICUNA para la implementación de redes neuronales

Carmen Calvo Cano

Resumen— Este trabajo evalúa el rendimiento del coprocesador vectorial Vicuna, basado en la extensión RISC-V Vector (RVV), para la aceleración de operaciones fundamentales en redes neuronales dentro de sistemas embebidos, especialmente en el contexto espacial del New Space. Para ello, se implementan diversos kernels (VVADD, SAXPY, REDUCE y MATMUL) mediante intrínsecos RVV, garantizando control explícito sobre SEW, LMUL y VL, y se analizan bajo múltiples configuraciones del ancho vectorial (VREG_W). Los resultados muestran mejoras crecientes al ampliar el paralelismo vectorial, aunque todos los kernels quedan limitados por el ancho de banda de memoria debido a la ausencia de caché en la simulación RTL. MATMUL, pese a su mayor intensidad aritmética, tampoco alcanza la región compute-bound, confirmando que el techo de rendimiento lo impone la memoria. En conjunto, el estudio demuestra el correcto funcionamiento de RVV sobre Vicuna, cuantifica la influencia de la configuración vectorial y señala la necesidad de aumentar el rendimiento de memoria para obtener aceleraciones superiores en aplicaciones basadas en redes neuronales.

Palabras clave— RISC-V, Extensión Vectorial, Vicuna, Redes Neuronales, RVV, CNN, Computación Embebida

Abstract— This work evaluates the performance of the Vicuna vector coprocessor, based on the RISC-V Vector Extension (RVV), for accelerating neural-network-related computations in embedded systems, with a focus on spaceborne processing in the New Space context. Several kernels (VVADD, SAXPY, REDUCE, and MATMUL) are implemented using RVV intrinsics to retain explicit control over SEW, LMUL, and VL, and are tested under multiple vector-width configurations (VREG_W). Results show increasing speedups as vector width grows, although all kernels remain limited by memory bandwidth because cache cannot be enabled in the RTL simulation environment. Even MATMUL, despite its higher arithmetic intensity, does not reach the compute-bound region, confirming that memory throughput sets the performance ceiling. Overall, the study validates the correct execution of RVV on Vicuna, quantifies the impact of vector-width scaling, and highlights the need for improved memory performance to achieve larger accelerations in neural-network workloads.

Index Terms— RISC-V, Vector Extension, Vicuna, Neural Networks, RVV, CNN, Embedded Computing

1 Introducción – Contexto del trabajo

El entorno espacial plantea grandes retos a nivel de computación *embedded*. La limitada capacidad energética y la imposibilidad de depender de enlaces de comunicación constantes hacen necesario desarrollar sistemas capaces de ejecutar algoritmos complejos directamente a bordo de satélites y plataformas espaciales. En este contexto surge el paradigma del *New Space*, que propone aprovechar componentes comerciales (*Commercial Off-The-Shelf,* COTS) para reducir costes y acelerar la innovación tecnológica [1]. En los últimos años, la arquitectura **RISC-V** ha adquirido relevancia al ser un estándar abierto, flexible y extensible, que permite a investigadores y empresas diseñar procesadores adaptados a necesidades específicas sin las restricciones impuestas por licencias propietarias [2]. Entre sus extensiones más destacadas se encuentra la **Vectorial (RVV)**, que habilita la ejecución de operaciones SIMD (Single Instruction, Multiple Data) directamente en CPU, ofreciendo un rendimiento comparable al de GPUs o NPUs, pero con menor consumo y mayor simplicidad de integración [3].

La computación vectorial se muestra especialmente útil en aplicaciones de **redes neuronales convolucionales (CNNs)**, ampliamente utilizadas en tareas de clasificación y procesado de imágenes satelitales [4]. Estas técnicas permiten, por ejemplo, filtrar imágenes nubladas antes de transmitirlas a tierra, optimizando así el ancho de banda y aumentando la eficiencia de la misión. Para validar el potencial de RVV en este tipo de escenarios, se ha optado por implementar y evaluar funciones básicas de álgebra lineal (SAXPY, REDUCE, VVADD, MATMUL y CONVOLUCIÓN), que representan la base del cálculo en la mayoría de los modelos de Deep Learning. Posteriormente, estas optimizaciones se trasladan a la ejecución de redes neuronales sobre **TensorFlow Lite** [5], utilizando plataformas comerciales que incorporan procesadores RISC-V con soporte para la extensión vectorial.

Sin embargo, muchos procesadores presentan una arquitectura cerrada, limitando así el estudio del impacto real que aportan las variaciones de hardware, como el tamaño de palabra, en el rendimiento. En este proyecto se aborda este vacío utilizando el procesador **Vicuna** [6], un procesador configurable en el que es posible modificar algunos parámetros de hardware. Gracias a esta flexibilidad, destaca frente a otros procesadores usados en proyectos similares.

El **objetivo** de este trabajo es, por lo tanto, **explorar el**

impacto de la extensión vectorial en la aceleración de redes neuronales sobre hardware embebido RISC-V, contribuyendo al desarrollo de soluciones viables para aplicaciones espaciales dentro del marco del New Space.

El proceso de ejecución de este proyecto estará dividido en 4 fases diferenciadas:

- Fase I. Estudiar el estado del arte: revisión de fundamentos teóricos de RISC-V y el uso del procesador Vicuna.
- Fase II. Entender el entorno de trabajo: comprensión y configuración del ecosistema de desarrollo del procesador Vicuna y las herramientas necesarias para la simulación y ejecución de código vectorial.
- Fase III. Crear entorno de pruebas y proceso de automatización sobre las mismas: análisis del rendimiento bajo distintas configuraciones de hardware.
- Fase IV. Valorar rendimiento en función de la arquitectura: recopilación de resultados y valoración de estos.

2 Marco Teórico

En este apartado se presentan los conceptos fundamentales que sustentan el trabajo: la arquitectura **RISC-V**, su extensión **Vectorial (RVV)** y el procesador **Vicuna**, destacando el valor que este último aporta al objetivo del proyecto.

2.1 Estado del Arte

La computación vectorial ha sido tradicionalmente empleada en arquitecturas de alto rendimiento, como supercomputadores y procesadores gráficos, para acelerar operaciones masivamente paralelas mediante el paradigma SIMD (Single Instruction, Multiple Data). Arquitecturas como x86 (con extensiones AVX) y ARM (con NEON) han incorporado instrucciones vectoriales para mejorar el rendimiento en aplicaciones de procesamiento multimedia, cálculos científicos y aprendizaje automático. Asimismo, aceleradores específicos como las GPUs y TPUs han demostrado una gran capacidad para ejecutar redes neuronales y tareas intensivas en datos, aunque presentan limitaciones en entornos embebidos debido a su elevado consumo energético y complejidad de integración [7].

En este contexto surge **RISC-V**, una arquitectura de conjunto de instrucciones abierta y modular que permite la incorporación de extensiones opcionales. Entre ellas destaca la extensión vectorial, ratificada en su versión 1.0 como estándar , que añade 186 nuevas instrucciones, 32 registros vectoriales y parámetros configurables, ofreciendo flexibilidad para adaptar el hardware a diferentes tipos de datos y tamaños de vector. Esta extensión busca competir con soluciones propietarias, proporcionando una alternativa abierta para optimizar aplicaciones de alto rendimiento en plataformas de bajo coste y consumo.

Actualmente, el ecosistema de **RVV** se encuentra en una fase temprana de adopción. Existen pocas implementaciones comerciales, como los procesadores Xuantie C910/C908 de T-Head y plataformas como BeagleV-Ahead o Banana Pi F3, que integran unidades vectoriales con longitudes de registro entre 128 y 256 bits. Estas soluciones han sido utilizadas en proyectos orientados a *edge computing* y aplicaciones espaciales, donde se busca reducir la dependencia de enlaces de comunicación mediante el procesamiento local de datos [8]. Sin embargo, los estudios realizados muestran que, aunque las operaciones básicas (SAXPY, MATMUL, CONVOLUCIÓN) alcanzan mejoras significativas en entornos controlados, la ganancia global en redes neuronales completas es más modesta (SpeedUp entre 1.7x y 2.5x), debido a cuellos de botella en memoria y limitaciones en el soporte de compiladores .

Por otro lado, la investigación en procesadores vectoriales configurables ha dado lugar a propuestas como **Vicuna**, un coprocesador RISC-V diseñado para garantizar predictibilidad temporal en aplicaciones críticas. Vicuna elimina características que generan anomalías temporales (como ejecución fuera de orden) y permite modificar parámetros de hardware para evaluar el impacto en rendimiento, manteniendo una eficiencia superior al 90 % en tareas compute-bound. Este enfoque resulta especialmente relevante en entornos donde la estimación del Worst-Case Execution Time (WCET) es esencial, como sistemas empotrados para el espacio o automoción.

En resumen, el estado del arte evidencia que la extensión vectorial de RISC-V representa una oportunidad para democratizar la computación paralela en dispositivos embebidos, aunque persisten retos relacionados con la optimización del acceso a memoria, la integración en frameworks como TensorFlow Lite y la falta de soporte maduro en compiladores. Estas limitaciones motivan la exploración de arquitecturas configurables como Vicuna, que aportan flexibilidad y predictibilidad en escenarios donde el rendimiento y la seguridad temporal son críticos.

2.2 RISC-V y RVV

RISC-V es una arquitectura abierta y modular que permite añadir extensiones opcionales al conjunto de instrucciones base, facilitando la creación de procesadores personalizados sin restricciones de licencias propietarias. Esta flexibilidad ha impulsado su adopción en entornos embebidos y plataformas de investigación, donde la posibilidad de adaptar el hardware al tipo de aplicación es especialmente valiosa para estudios de rendimiento y variación arquitectónica.

Entre estas extensiones destaca RVV (RISC-V Vector Extension), diseñada para proporcionar un modelo SIMD escalable que permite procesar múltiples datos en paralelo dentro de la propia CPU. RVV introduce 32 registros vectoriales y parámetros configurables como **SEW** (tamaño del dato), **LMUL** (agrupación de registros) y **VL** (número de elementos procesados por iteración). Estos parámetros permiten ajustar el paralelismo efectivo en función del tipo de dato y de la longitud física del vector del procesador (**VLEN**), manteniendo la portabilidad del código incluso cuando el hardware cambia. Este enfoque contrasta con arquitecturas con vectores de tamaño fijo, y resulta especialmente útil para evaluar cómo influyen distintas configuraciones de hardware en el rendimiento. Para trabajar con RVV en C, se emplean **intrínsecos**, funciones que

representan directamente instrucciones vectoriales y permiten controlar explícitamente SEW, LMUL y VL. Su uso garantiza la generación de instrucciones RVV, algo especialmente importante porque los compiladores actuales aún presentan limitaciones para autovectorizar bucles complejos. Por este motivo, en este trabajo los kernels se implementan mediante intrínsecos, asegurando así un control preciso sobre la vectorización y una evaluación fiable del impacto de las configuraciones vectoriales del procesador[9].

Los **intrínsecos RVV** son funciones del lenguaje C que representan directamente instrucciones vectoriales de la extensión RVV del estándar RISC-V. A diferencia del código C tradicional, donde el compilador decide si vectorizar o no un bucle, los intrínsecos permiten **invocar explícitamente** operaciones vectoriales como cargas, sumas, multiplicaciones o reducciones, garantizando que el binario generado incluya instrucciones RVV. Su uso es especialmente útil en entornos donde la **autovectorización** [10] del compilador aún es **limitada**, como sucede actualmente con RVV en GCC/LLVM, y donde se requiere controlar SEW, LMUL y VL durante la ejecución. Gracias a los intrínsecos, es posible adaptar el comportamiento del kernel a las distintas configuraciones hardware del procesador, asegurando una evaluación más precisa del rendimiento vectorial.

2.3 Vicuna

Vicuna es **un coprocesador vectorial RISC-V** de código abierto diseñado para implementar la extensión Zve32x, una variante ligera de RVV orientada a sistemas embebidos que trabajan únicamente con enteros de 8, 16 y 32 bits. Funciona como un acelerador conectado a un procesador principal (como Ibex o CV32E40X) mediante la interfaz CORE-V-XIF [11], delegando en él las operaciones vectoriales mientras la CPU continúa ejecutando el flujo escalar.

Su arquitectura ha sido diseñada para ser **temporalmente predecible**, evitando mecanismos como ejecución fuera de orden o especulación, lo que facilita el análisis de rendimiento y la repetibilidad en entornos críticos.

Una de las características centrales de Vicuna es su alto grado de **configurabilidad**, que permite ajustar parámetros internos del hardware. Entre los más relevantes se encuentran:

- VREG_W. Define el ancho lógico de los registros vectoriales (por ejemplo 128, 256, 512 o más bits)
- VMEM_W. Define la anchura de la interfaz de memoria
- ICACHE/DCACHE. Define el tamaño de la caché de instrucciones y datos respectivamente
- El número y tipo de pipelines vectoriales

Vicuna incorpora varias unidades funcionales especializadas como VLSU (load/store), VALU (operaciones aritméticas), VMUL (multiplicación), VSLD (desplazamientos) y VELEM (permutaciones y reducciones). Estas unidades se integran dentro de un pipeline vectorial configurable. No obstante, aunque forman parte fija del diseño, la arquitectura permite modificar parámetros como el número de pipelines, su anchura y la organización interna, lo que influye directamente en cómo se explotan estas unidades durante la ejecución.

Gracias a esta modularidad, el procesador puede adaptarse tanto a kernels *memory-bound* como SAXPY/VVADD como a kernels *compute-bound* como MATMUL, modificando el paralelismo interno sin cambiar el código fuente de los programas.

En el contexto de este trabajo, Vicuna resulta especialmente adecuado porque permite estudiar el impacto real de la configuración del hardware vectorial en el rendimiento: variando VREG_W, experimentando con diferente granularidad de VL y analizando cómo cada ajuste afecta al número de ciclos. Su diseño determinista y la posibilidad de ejecutar simulaciones RTL reproducibles lo convierten en una plataforma ideal para evaluar de forma controlada los kernels implementados mediante intrínsecos RVV, proporcionando un entorno mucho más flexible que las implementaciones comerciales actuales de RVV.

3 METODOLOGÍA

La metodología seguida para evaluar el comportamiento del coprocesador vectorial Vicuna bajo diferentes configuraciones y kernels representativos de redes neuronales. La metodología se ha diseñado para garantizar un entorno controlado, reproducible y totalmente parametrizable, permitiendo aislar el impacto de elementos clave como el ancho vectorial, el tipo de dato y la granularidad del procesamiento vectorial. Para ello, el trabajo se divide en cuatro etapas: análisis del repositorio y herramientas, implementación de los kernels vectoriales mediante intrínsecos RVV, y la automatización del proceso de simulación y medida de rendimiento.

3.1 Análisis del entorno Vicuna y organización del repositorio

Para desarrollar las pruebas de este trabajo se ha utilizado **Vicuna**, un coprocesador vectorial RISC-V cuyo repositorio oficial está estructurado para facilitar tanto la **simulación RTL** como la ejecución de software bare-metal [12]. El proyecto se trabaja íntegramente en **simulación**, sin desplegarse sobre hardware físico, lo que permite evaluar diferentes configuraciones del procesador de forma reproducible y sin restricciones derivadas de plataformas específicas. Vicuna se distribuye junto a un conjunto de herramientas, scripts y Makefiles que automatizan la construcción del coprocesador y la ejecución de programas de prueba, generando en cada simulación una versión del hardware ajustada a los parámetros seleccionados.

El repositorio está organizado en varios directorios relevantes para este TFG. La carpeta **rtl/** contiene la implementación del coprocesador vectorial, incluyendo el registro vectorial, las unidades funcionales, los pipelines y la lógica de control. La carpeta **sim/** incluye los distintos entornos de simulación (como Verilator) junto con los Makefiles responsables de generar el hardware configurado y ejecutar los binarios. Por su parte, la carpeta **sw/** contiene el código en C utilizado para compilar los programas que se ejecutarán sobre Vicuna, así como plantillas iniciales que permiten incluir versiones vectoriales y escalares de los kernels.

La **parametrización** del coprocesador se gestiona principalmente a través de los Makefiles proporcionados en el repositorio. Entre ellos destaca **config.mk**, donde se definen los valores que condicionan la configuración del hardware antes de la simulación.

3.2 Preparación del entorno y herramientas utilizadas

Para garantizar un acceso estable y seguro al entorno de simulación, el proyecto se desarrolla de manera remota sobre una máquina de la UAB. Para ello se utiliza **FortiClient** VPN, que permite establecer una conexión cifrada con la red institucional, y **MobaXterm**, empleado como terminal SSH para acceder al servidor, transferir archivos y ejecutar las simulaciones. En primer lugar, se descargan las herramientas en el dispositivo y se configuran. FortiClient se conecta a la **VPN** de la Universidad Autónoma de Barcelona mediante las credenciales universitarias, accediendo al Gateway remoto *xvp.uab.cat*, puerto 443. Una vez establecida la conexión, se accede a MobaXterm y se configura una conexión remota a la máquina que se utilizará para las simulaciones. A continuación, tras configurar las herramientas se clona el repositorio de github en local para poder trabajar con Vicuna.

El entorno de ejecución se basa en herramientas estándar del ecosistema RISC-V, incluyendo la toolchain bare-metal para compilar **programas en C** y el simulador **Verilator**, utilizado para ejecutar el coprocesador Vicuna a nivel RTL. Esta combinación permite compilar los kernels vectoriales, generar el binario correspondiente y ejecutarlo directamente sobre una instancia simulada del hardware. El uso de **simulación RTL** resulta especialmente adecuado en este proyecto, ya que ofrece una monitorización precisa del comportamiento del coprocesador bajo distintas configuraciones sin necesidad de disponer de hardware físico.

Con esta preparación, el entorno queda listo para compilar, ejecutar y analizar sistemáticamente los kernels utilizados en el estudio, garantizando reproducibilidad en todas las pruebas y evitando variaciones derivadas del sistema operativo o del hardware del usuario.

3.3 Configuración del procesador y parametrización de las simulaciones

La simulación de Vicuna permite ajustar distintos **parámetros** de la arquitectura antes de ejecutar cada programa, de modo que es posible generar configuraciones de hardware adaptadas a cada experimento **sin modificar el código RTL**. Esta configuración se gestiona mediante las variables definidas en los **Makefiles** del repositorio, entre las cuales destacan aquellas que afectan directamente al comportamiento del coprocesador vectorial.

El parámetro más relevante en este proyecto es **VREG_W**, que establece el ancho lógico de los registros vectoriales. Su valor determina cuántos bits puede procesar Vicuna por iteración y, en consecuencia, cuántos elementos se procesan simultáneamente en una iteración en función de SEW. Variar este parámetro permite estudiar cómo cambia el rendimiento al modificar la **granularidad** del paralelismo: anchos pequeños generan más iteraciones del bucle vectorial, mientras que anchos mayores reducen el número de pasadas del **strip-mining**.

Asimismo, durante la compilación del software se utiliza el parámetro **VEC**, que activa o desactiva el soporte vectorial en el binario. Esto permite generar automáticamente dos versiones equivalentes de cada kernel (una escalar y otra vectorial) manteniendo el mismo código fuente. Gracias a ello, las comparaciones entre ambas variantes se realizan en condiciones idénticas.

Una vez fijados estos parámetros, la simulación genera la configuración correspondiente del coprocesador, carga el binario en memoria y ejecuta el programa hasta alcanzar la instrucción final.

3.4 Diseño de los Kernels

Los **kernels** diseñados en este trabajo siguen una estructura común que permite evaluar de manera coherente el impacto de la vectorización en diferentes patrones de cómputo. Cada kernel incluye dos rutas de ejecución: una **versión escalar**, compilada sin soporte vectorial, y una **versión vectorial** basada en intrínsecos RVV. Esta unificación dentro del mismo código facilita comparar ambas variantes bajo las mismas condiciones, ya que únicamente cambia la activación del parámetro **VEC** durante la compilación.

La implementación vectorial utiliza intrínsecos RVV para controlar explícitamente aspectos como el tamaño del elemento (**SEW**), la agrupación de registros (**LMUL**) y el número de elementos procesados en cada iteración mediante ***vsetvl***. Todos los kernels siguen un patrón de *strip-mining*: el vector se procesa por bloques cuyo tamaño se ajusta dinámicamente en función de SEW, LMUL y el número de elementos restantes. De esta manera, el mismo código es válido para cualquier valor de VREG_W, lo que resulta esencial para estudiar cómo cambia el rendimiento al modificar el ancho vectorial del coprocesador.

Los kernels seleccionados avanzan en complejidad de forma progresiva. El primero, **VVADD**, aplica una suma elemento a elemento y sirve para validar que el flujo vectorial funciona correctamente y que las instrucciones RVV aparecen en el ensamblador. A continuación, **SAXPY** introduce una multiplicación escalar adicional, manteniendo un patrón *element-wise* pero con mayor trabajo por elemento. El kernel **REDUCE** modifica la estructura del cómputo incorporando una reducción horizontal, lo que permite evaluar un tipo distinto de dependencia y el uso de instrucciones como *vredsum*. Finalmente, **MATMUL** representa el escenario de mayor intensidad aritmética, con tres bucles anidados y un volumen de operaciones significativamente superior. Esta progresión permite analizar, de forma controlada, cómo afecta la vectorización a kernels desde claramente *memory-bound* hasta otros más cercanos al *compute-bound*.

3.5 Medición de rendimiento

Para analizar el comportamiento del coprocesador Vicuna se emplea un sistema de medición basado en el contador de ciclos del procesador, accesible a través de la función **get_mcycle()**. Esta función sirve para leer el contador de ciclos del procesador RISC-V, es decir, cuántos ciclos de reloj han pasado desde que se inició la ejecución. Antes y

después de cada kernel se lee este registro, y se restan los resultados para obtener los ciclos transcurridos (**Figura 1**).

$$Speedup = \frac{Tiempo_{No\ vectorial}}{Tiempo_{Vectorial}} \quad (1)$$

$$Mejora\ (\%) = \frac{Tiempo_{No\ vectorial} - Tiempo_{Vectorial}}{Tiempo_{No\ vectorial}} \times 100 \quad (2)$$

A partir de las mediciones obtenidas se calculan métricas como el **SpeedUp** (1) y la **mejora porcentual** (2), comparando los ciclos de la versión escalar y la vectorial bajo condiciones idénticas. Estas métricas permiten identificar el comportamiento *memory-bound* o *compute-bound* de cada kernel y analizar cómo influyen parámetros como el tamaño del problema o el ancho vectorial configurado. En conjunto, este proceso proporciona una base cuantitativa sólida para evaluar el impacto de la extensión RVV en el rendimiento de los kernels estudiados.

3.6 Automatización, simulación RTL y proceso de ejecución

Las **pruebas de rendimiento** se ejecutan mediante un **flujo automatizado** que permite compilar los kernels, generar el binario correspondiente y lanzar la simulación RTL sin intervención manual. Este proceso se articula a través de los **Makefiles** del repositorio. De este modo, es posible realizar múltiples ejecuciones consecutivas variando únicamente los parámetros de interés, como VREG_W o el tamaño de los datos procesados.

Algorithm 1: test inicial con strip-mining (RVV)

```
Input: Arreglo fuente array[0.,15] (int8)
Output: Arreglo destino target[0.,15] (int8)
Inicialización
    src ← &array[0],  dst ← &target[0]
    n ← sizeof(array)
    tt_ini ← get_mcycle()
Bucle principal (strip-mining por VL)
while n > 0 do
    vl ← vsetvl_e8m1(n)                 // Fija SEW=8, LMUL=1 y VL
    v⃗ ← vle8(src, vl)                   // Carga contigua de 8-bit
    v⃗ ← vmul_vx_i8m1(v⃗, 5, vl)        // Multiplica cada elem. por 5
    v⃗ ← vadd_vx_i8m1(v⃗, 7, vl)        // Suma escalar 7 a cada elem.
    vse8(dst, v⃗, vl)                    // Almacena contiguo
    src ← src +vl;  dst ← dst +vl;  n ← n − vl
Medición y volcado
    tt_end ← get_mcycle()
Fin
    Saltar a dirección 0 (fin de simulación)
```

Figura 1. Pseudocódigo que implementa una operación vectorial sobre un array usando intrínsecos RVV (multiplicación y suma escalar) y mide el tiempo de ejecución en ciclos. El programa se ejecuta hasta llegar al salto a dirección 0, que provoca una interrupción.

Durante la **simulación**, el binario generado se carga en la memoria de programa del modelo RTL y el coprocesador comienza a ejecutar las instrucciones desde la dirección inicial. El flujo continúa de forma secuencial hasta alcanzar la instrucción final del programa, que desencadena una **interrupción** (**Figura 1**) predefinida y marca el final de la ejecución. Este comportamiento reproduce el funcionamiento típico de un sistema bare-metal, eliminando dependencias externas y asegurando que los tiempos medidos reflejan únicamente la actividad interna del procesador.

La automatización permite además realizar **barridos** sistemáticos, ejecutando de manera iterativa distintas combinaciones de parámetros. Esto facilita el análisis comparativo entre configuraciones y garantiza que todas las simulaciones se realizan bajo condiciones homogéneas. Gracias a este enfoque, es posible **obtener métricas** consistentes para cada kernel y **estudiar cómo varían** en función del ancho vectorial, la longitud del problema o la naturaleza del cómputo, manteniendo un control total sobre el entorno experimental.

4 Resultados

En este apartado se describen los Kernels utilizados, los resultados obtenidos y el análisis de los mismos.

4.1 Kernel VVADD

El primer kernel evaluado es **VVADD** (z = x + y), una operación elemental que suma dos vectores posición a posición. Este kernel constituye el punto de partida por dos razones: su estructura es extremadamente simple (operación aritmética *element-wise*) y permite verificar que el coprocesador Vicuna ejecuta correctamente instrucciones RVV, y al mismo tiempo proporciona una referencia base del comportamiento de un kernel claramente *memory-bound*.

En la versión vectorial, VVADD utiliza cargas contiguas y una única operación aritmética por elemento (suma con *vadd*), de modo que el coste computacional es muy reducido y el rendimiento depende casi por completo del acceso a memoria. Este patrón permite observar con claridad el impacto del ancho vectorial **VREG_W**. Cuando el ancho vectorial es reducido, el bucle vectorial requiere varias iteraciones, acumulando el *overhead* fijo asociado a cada llamada a *vsetvl*. A medida que aumenta, se reduce el número de iteraciones necesarias y la mejora se vuelve más evidente, aunque limitada por la propia presión sobre memoria.

Los **resultados** obtenidos confirman este comportamiento. Cuando VREG_W es pequeño (128), y los vectores también (4-16 elementos) no se refleja la capacidad vectorial, ya que el coste fijo de cada iteración limita el rendimiento. Es decir, se obtienen menos iteraciones, pero mucho más costosas que en escalar, afectando al rendimiento. Para un tamaño de 4096 elementos (los vectores grandes dan resultados más significativos ya que se amortiza el *overhead*), la versión escalar dura **≈151,6 K ciclos**, frente a **≈116,3 K ciclos** de la versión vectorial. Obtenemos una reducción significativa en el tiempo de ejecución. Por último, aumentar el ancho vectorial VREG_W la mejora crece rápidamente: **23%** con 128 bits hasta **79%** con 1024 bits. Esta tendencia refleja una disminución del número de iteraciones del bucle vectorial y demuestra que el ***overhead*** inicial queda plenamente **amortizado** cuando se dispone de un ancho vectorial suficiente. No obstante, el hecho de que el SpeedUp no crezca indefinidamente (**Figura 2**) confirma que VVADD sigue siendo un kernel *memory-bound*, **limitado por el ancho de banda** y no por la capacidad de cómputo de la unidad vectorial. En conjunto, estos

resultados **validan** el correcto **funcionamiento** de RVV en Vicuna y establecen un punto de referencia claro para interpretar los kernels posteriores de mayor complejidad.

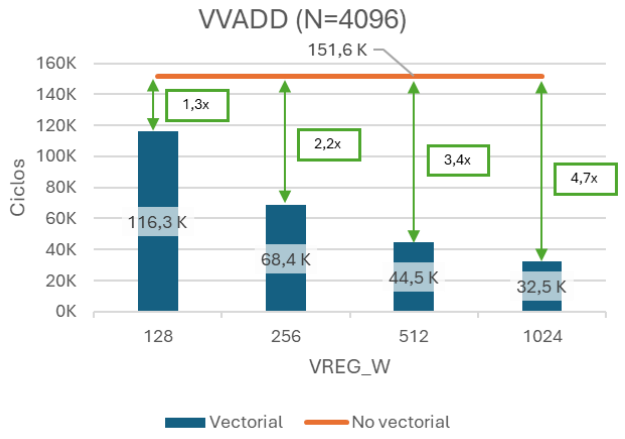

Figura 2. Ciclos de ejecución del kernel VVADD (N = 4096) para distintas configuraciones de **VREG_W**. La versión vectorial reduce progresivamente el número de ciclos a medida que aumenta el ancho vectorial, alcanzando SpeedUps cada vez mayores frente a la versión escalar. A pesar de ello, el SpeedUp se saturará, aunque se aumente el ancho vectorial.

4.2 Kernel SAXPY

El kernel **SAXPY** (z = a·x + y) introduce una operación adicional respecto a VVADD (una multiplicación escalar), pero sigue siendo un patrón *element-wise* cuyo rendimiento está ampliamente condicionado por el acceso a memoria. Por este motivo, aunque SAXPY incorpora más cómputo por elemento, continúa siendo un kernel fundamentalmente *memory-bound*, y su comportamiento depende de la capacidad del coprocesador para amortizar el coste fijo asociado a cada iteración vectorial.

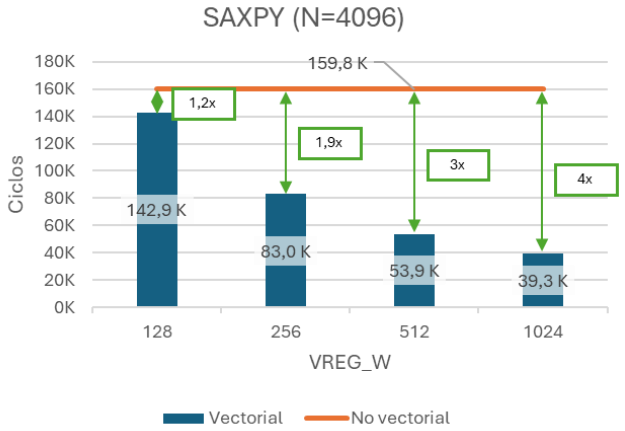

Figura 3. Ciclos de ejecución del kernel SAXPY (N = 4096) para distintas configuraciones de **VREG_W**. La versión vectorial reduce el tiempo de ejecución a medida que aumenta el ancho vectorial, alcanzando SpeedUps cada vez mayores frente a la versión escalar. Aunque la mejora es clara, el rendimiento sigue limitado por el ancho de banda de memoria (a pesar de aumentar la intensidad aritmética), lo que explica que los SpeedUps sean ligeramente inferiores a los obtenidos en VVADD, ya que hay más operaciones por iteración.

Los **resultados** obtenidos (**Figura 3**) muestran que, para un tamaño de 4096 elementos, la versión escalar requiere aproximadamente **159,8 K ciclos**, mientras que la versión vectorial reduce este tiempo dependiendo de VREG_W. Con un ancho vectorial de 128 bits, el SpeedUp es de **1,2x**, ligeramente inferior al observado en VVADD bajo las mismas condiciones. Este patrón vuelve a repetirse para 256 bits, donde SAXPY alcanza un SpeedUp de **1,9x**, también por debajo del **2,2x** obtenido en VVADD.

A medida que aumenta VREG_W, la mejora se vuelve más significativa. SAXPY alcanza un SpeedUp de **4x** para **1024 bits**, manteniendo la tendencia creciente esperada a medida que disminuye el número de iteraciones del bucle vectorial. Sin embargo, incluso en estos casos, la mejora sigue siendo algo inferior a la de VVADD, lo que confirma que la multiplicación adicional no basta para desplazar el kernel fuera de la región *memory-bound*. En consecuencia, el paralelismo disponible se aprovecha parcialmente, pero está limitado por el ancho de banda de memoria, no por la capacidad de cómputo del coprocesador.

En conjunto, SAXPY presenta una mejora progresiva y clara con el incremento del ancho vectorial, pero también pone de manifiesto que, en kernels dominados por memoria, la presencia de operaciones adicionales no garantiza un mayor SpeedUp. Este comportamiento anticipa la transición hacia kernels con dependencias internas, como **REDUCE**, donde el patrón de ejecución deja de ser puramente *element-wise* y las limitaciones cambian por completo.

4.3 Kernel REDUCE

El kernel **REDUCE** (acc = x + y) realiza la multiplicación elemento a elemento de dos vectores y acumula los resultados en un único valor final. A diferencia de VVADD y SAXPY, deja de ser un patrón puramente *element-wise* y pasa a incluir una **dependencia horizontal** debida a la fase de reducción, implementada mediante instrucciones como vredsum. Además, esta operación introduce un ensanchamiento de precisión (16x16 → 32 bits), lo que añade trabajo adicional por iteración y modifica el comportamiento del kernel respecto a los casos anteriores.

REDUCE permite observar con claridad la **granularidad interna de la VPU de Vicuna**, ya que el coprocesador procesa bloques fijos de elementos en función de SEW y VREG_W. Cuando el tamaño del vector es menor o igual que dicho bloque interno, el tiempo de ejecución apenas varía, ya que en una sola iteración se procesan todos los elementos. Asimismo, este kernel combina movimiento de datos y reducción, lo que lo aproxima más a patrones reales empleados en redes neuronales, donde la agregación de resultados forma parte de operaciones como convoluciones o acumulaciones parciales.

Los resultados obtenidos muestran que, con *VREG_W = 128 bits* (**Figura 4**), la versión vectorial es **más lenta que la escalar**, incluso para vectores relativamente grandes (N ≈ 32–64). Esto se debe a que, con un ancho vectorial reducido, la VPU debe ejecutar muchas iteraciones del *strip-mining*, acumulando el ***overhead* fijo** asociado a cada llamada a *vsetvl* y a las fases de reducción parcial. Dado que REDUCE introduce dependencias horizontales más costosas que las operaciones *element-wise*, este coste fijo domina la

ejecución y la vectorización no compensa hasta que el volumen de datos es suficientemente grande.

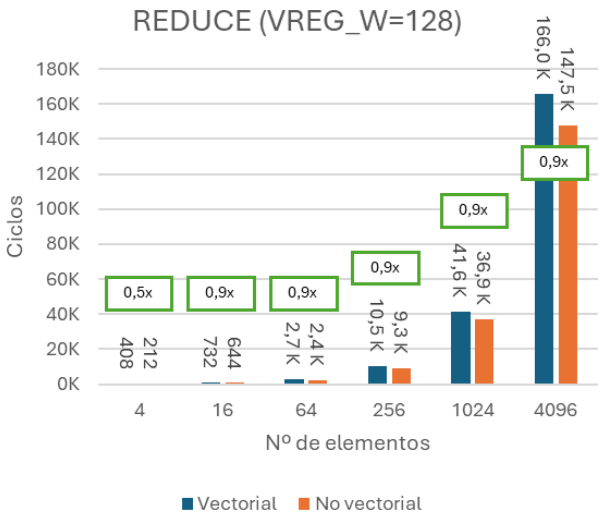

Figura 4. Ciclos de ejecución del kernel REDUCE para distintas longitudes de vector con **VREG_W = 128 bits**. La versión vectorial no supera a la escalar debido al elevado coste fijo por iteración y a las dependencias internas de la reducción, dando lugar a SpeedUps inferiores a 1 incluso para tamaños grandes. Este comportamiento confirma que, con anchos vectoriales estrechos, el *overhead* domina sobre el paralelismo disponible.

En contraste, al aumentar *VREG_W = 256-1024 bits* el comportamiento cambia. Con anchos vectoriales mayores, el número de elementos procesados en cada iteración crece y se reduce el número de reducciones parciales necesarias, lo que mejora el rendimiento y aproxima los tiempos vectoriales a los escalares. Para vectores pequeños de 4, 16 e incluso 64 elementos (este último solo para un ancho vectorial de 1024 bits), se obtienen los mismos ciclos (**Tabla 1**), ya que ambos casos procesan todos los datos en una única iteración (**Tabla 2**) debido al tamaño del bloque interno establecido por SEW y VREG_W.

Nº de elementos	**4**	**16**	**64**
VREG_W = 256 bits	480	480	1,7 K
VREG_W = 512 bits	624	624	1,2 K
VREG_W = 1024 bits	912	912	912

Tabla 1. Ciclos de ejecución del kernel REDUCE para distintas longitudes de vector con **VREG_W = 256, 512 y 1024 bits**. Cuando el tamaño del vector es menor o igual a VL, el tiempo se mantiene estable, ya que los costes fijos de la iteración predominan el tiempo de ejecución. Por ejemplo: los elementos son de SEW = 16 bits y el ancho vectorial es *VREG_W = 1024 bits*, obtenemos VLmax = 64 elementos, lo cual explica que para vectores con 64 o menos elementos el tiempo de ejecución viene definido por el mismo número de iteraciones, en este caso 1.

Nº de elementos	**4**	**16**	**64**
VREG_W = 256 bits	1	1	4
VREG_W = 512 bits	1	1	2
VREG_W = 1024 bits	1	1	1

Tabla 2. Iteraciones del bucle *strip-mining* del kernel REDUCE para distintas longitudes de vector con **VREG_W = 256, 512 y 1024 bits**. Para un ancho vectorial de 256 bits VLmax = 16 elementos, para 512 bits VLmax = 32 elementos y para 1024 bits VLmax = 64 elementos.

En conjunto, REDUCE muestra que la vectorización solo resulta beneficiosa cuando el ancho vectorial es suficientemente grande como para minimizar el número de reducciones parciales (**Figura 5**). Para configuraciones estrechas, el coste fijo domina y la versión escalar resulta más eficiente; para configuraciones amplias, el rendimiento mejora, aunque sin alcanzar los SpeedUps obtenidos en VVADD o SAXPY debido a las dependencias internas del kernel. De esta manera, REDUCE actúa como un punto intermedio entre los patrones *element-wise* y los kernels de mayor intensidad computacional como **MATMUL**.

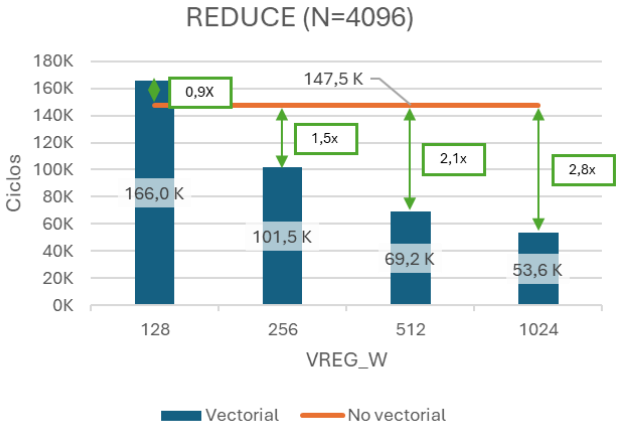

Figura 5. Ciclos de ejecución del kernel REDUCE (N = 4096) para distintas configuraciones de **VREG_W**. La versión vectorial reduce el tiempo de ejecución a medida que aumenta el ancho vectorial, alcanzando SpeedUps cada vez mayores frente a la versión escalar. Con **VREG_W = 128 bits**, el *overhead* de la reducción mantiene el rendimiento por debajo del escalar (0,9×). La tendencia confirma que, para este kernel, la vectorización solo resulta efectiva cuando se procesan bloques amplios por iteración.

4.4 Kernel Matmul

El kernel **MATMUL** (C = A × B) se implementa en modo vectorial mediante productos escalares sobre la fila de A y la columna de B. Para maximizar la continuidad de datos y el aprovechamiento de la VLSU, B se **traspone previamente** en un búfer B_T, de manera que cada "columna" de B pasa a ser una fila contigua en B_T. Con ello, ambos operandos del *dot product* (A y B_T) se leen con cargas contiguas (*vle16*) y el núcleo aritmético combina multiplicación ancha (*vwmul* 16×16→32) con reducción (*vredsum*) para acumular en precisión extendida (*int32*). Este patrón elimina accesos no contiguos y reduce el coste fijo por iteración del *strip-mining* al aumentar VREG_W (menos

llamadas a *vsetvl*, menos saltos y mayor paralelismo por vector).

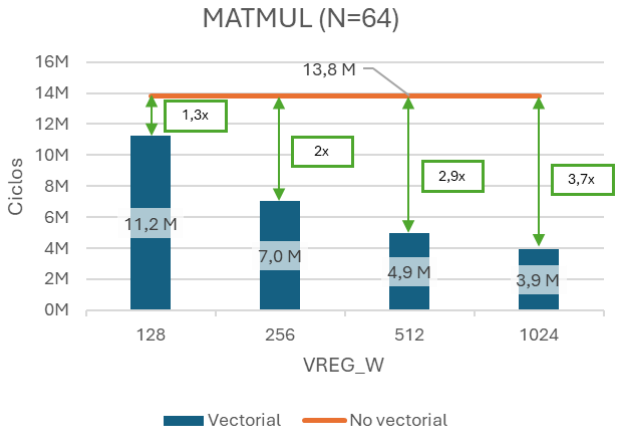

Figura 6. Ciclos de ejecución del kernel MATMUL (N = 64) para distintas configuraciones de **VREG_W**. La versión vectorial reduce el tiempo de ejecución a medida que aumenta el ancho vectorial, alcanzando SpeedUps cada vez mayores frente a la versión escalar. Debido a no poder usar caché, se pierde el posible rendimiento obtenido sobre un kernel con suficiente intensidad aritmética como para ser *compute-bound*.

A diferencia de VVADD y SAXPY (claramente *memory-bound*), MATMUL presenta **mayor intensidad aritmética** y se acerca más al comportamiento de redes neuronales. En la práctica, en el entorno de simulación utilizado (sin caché activable), el rendimiento de MATMUL **sigue limitado por memoria** para tamaños grandes, pero el uso de B_T permite un aprovechamiento más estable del vectorizado: a medida que crece VREG_W, el número de iteraciones por fila/columna disminuye y los ciclos bajan de forma monotónica hasta saturar por ancho de banda (**Figura 6**). En conjunto, MATMUL ofrece una mejora consistente frente a la versión escalar cuando el tamaño del problema es suficiente y el ancho vectorial es amplio, reproduciendo la misma pauta de "caída y saturación" observada en el resto de *kernels*, aunque con mejor correlación entre VREG_W y ciclos gracias a la mayor carga computacional por dato.

5 Limitaciones

En este apartado se describen las limitaciones encontradas a lo largo de la ejecución del trabajo.

5.1 La caché

Todas las medidas se han obtenido en simulación RTL sin ICACHE/DCACHE debido a bloqueos al activar la jerarquía de memoria en el repositorio de Vicuna. En ausencia de caché, la localidad de los accesos contiguos (por ejemplo, en B_T de MATMUL) no se materializa en mejoras significativas, y los kernels acaban definido por el techo de memoria (región *memory-bound* del roofline

Esto explica la saturación del SpeedUp al incrementar VREG_W incluso en kernels con mayor intensidad aritmética (como MATMUL) y la escasa separación entre acceso no contiguo y contiguo observada en tus métricas. El modelo roofline formaliza precisamente este límite: el rendimiento queda acotado por la pendiente del techo de memoria cuando la intensidad aritmética no permite cruzar a la región *compute-bound*.

5.2 Parámetros explorados

Este trabajo se centra en VREG_W y en el layout por su impacto directo en los kernels elegidos. Otros parámetros como **VMEM_W** (que no se puede modificar en consecuencia del fallo en el uso de la cache) o el número/organización de **pipelines** no se han variado sistemáticamente. Esto se debe a que van asociados a la implementación de cache.

Dado que los resultados apuntan a un límite por memoria, modificar VMEM_W permitiría elevar el techo de memoria del roofline (aumenta la pendiente del tramo diagonal) y observar si algunos kernels (en especial MATMUL con B_T) comienzan a acercarse a la región *compute-bound* (tramo horizontal).

5.3 Autovectorización

La autovectorización para RVV en compiladores generalistas (GCC/LLVM) todavía es limitada en bucles complejos; por ello, se han utilizado intrínsecos para garantizar la emisión de mnemónicos RVV, controlar SEW/LMUL/VL y aislar el efecto de VREG_W. Esto aumenta la trazabilidad del experimento, pero eleva el esfuerzo de programación y complica la generalización automática de optimizaciones.

El enfoque es coherente con el estado actual del ecosistema y con el uso de intrínsecos en evaluaciones de arquitecturas vectoriales predecibles como Vicuna (modelo y metodología de evaluación).

6 Conclusiones

A lo largo del desarrollo del trabajo se ha demostrado, con medidas sistemáticas en simulación RTL sobre Vicuna, que los kernels VVADD, SAXPY y REDUCE escalan con el ancho vectorial VREG_W, pero se saturan por memoria, incluso cuando el paralelismo interno aumenta (*VREG_W = 1024*). VVADD es el caso base con mejoras claras y sostenidas; SAXPY mantiene la tendencia, pero con SpeedUps algo inferiores; REDUCE es el más sensible al *overhead* de reducción (con VREG_W pequeños puede rendir peor que escalar) y solo mejora de forma apreciable cuando el bloque vectorial es grande. En MATMUL, con B traspuesta precomputada y sin *tiling*, el rendimiento también mejora con **VREG_W**, pero no entra en régimen *compute-bound*; el cómputo se estanca por la misma causa de fondo: el límite de la memoria. En conjunto, la VPU ejecuta correctamente RVV y el incremento de VREG_W reduce iteraciones y ciclos, pero el techo lo impone la jerarquía de memoria, no la capacidad de cómputo.

El análisis **roofline** (**Figura 7**) que se ha construido sitúa el pico de cómputo muy por encima del techo de memoria efectivo; el cambio de techo está en una IA (~169 ops/byte) muy superior a la IA de los kernels. Por este motivo, todos los puntos caen en la región *memory-bound* (pendiente limitada por el ancho de banda), de modo que aumentar

VREG_W ayuda a amortizar *overhead* y mejora el rendimiento, pero no cambia el techo. Esto explica que VVADD/SAXPY presenten mejoras crecientes pero saturadas, que REDUCE penalice con VREG_W estrecho (coste de reducciones parciales) y que MATMUL**, sin caché activa**, muestre una IA baja, quedándose también limitada por memoria.

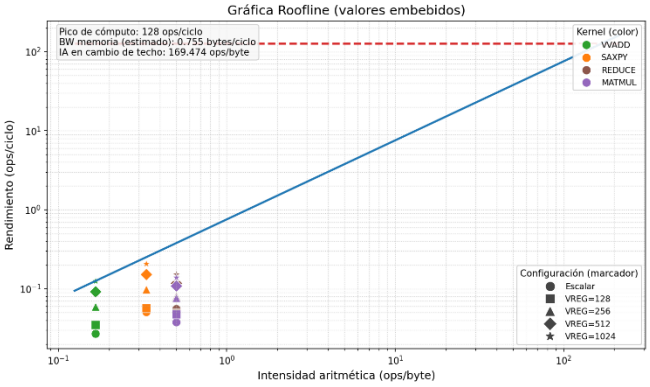

Figura 7. Roofline en dominio de ciclos. Pico de cómputo fijado en 128 ops/ciclo (línea horizontal roja) y techo de memoria estimado por la pendiente ≈ 0,755 bytes/ciclo (línea azul); el punto de cambio de techo se sitúa en IA ≈ 169,5 ops/byte. Los colores identifican el kernel (**VVADD** verde, **SAXPY** naranja, **REDUCE** marrón, **MATMUL** morado) y los marcadores la configuración). Todos los puntos quedan en región *memory-bound*: aumentar *VREG_W* reduce iteraciones y ciclos, pero no supera el límite impuesto por la memoria.

A pesar de la falta de tiempo para abordar este problema, si se pudiera usar la caché, habría un estudio mucho más extenso sobre todo en el kernel MATMUL. El layout junto con la caché (correctamente definidos) generarían una mejora considerable del rendimiento. La implicación directa es clara: para dar el siguiente salto se debería **elevar el techo de memoria** (habilitar caché/VMEM_W) y **subir la IA efectiva** (matrices más grandes) para acercarse a la región *compute-bound*. De esta manera podremos valorar el rendimiento ante **ejecuciones *compute-bound***.

6 Uso de la IA

Se ha utilizado inteligencia artificial generativa de forma acotada y transparente como apoyo en tareas auxiliares de elaboración del documento: organización y alineación de apartados, comprobación de coherencia entre secciones y verificación terminológica básica; además, para proponer títulos, pies de figura y leyendas más uniformes. La herramienta se ha empleado únicamente como asistencia editorial y de presentación, sin introducir contenido técnico nuevo ni modificar los planteamientos metodológicos.

No se ha recurrido a la IA para definir la metodología, diseñar experimentos, generar datos, programar las pruebas ni interpretar resultados. Las métricas, configuraciones, análisis y conclusiones técnicas proceden de las simulaciones y del trabajo realizado con Vicuna; la IA se ha limitado a un soporte no determinante orientado a la claridad y consistencia del texto final.

Bibliografía

[1] Thriving on Our Changing Planet: A Decadal Strategy for Earth Observation from Space, National Academies Press.

[2] RISC-V, «rvv-intrinsic-doc,» GitHub, [Online]. Available: https://github.com/riscv-non-isa/rvvintrinsic-doc.

[3] RISC-V, "Release vector extension 1.0, frozen for public review · riscv/riscv-v-spec" .

[4] Chen Chen, Xiaoyan Xiang, Chang Liu, Yunhai Shang, Ren Guo, Dongqi Liu, Yimin Lu, Ziyi Hao, Jiahui Luo, Zhijian Chen, Chunqiang Li, Yu Pu, Jianyi Meng, Xiaolang Yan, Yuan Xie, and Xiaoning Qi, "Xuantie-910: A commercial multi-core 12-stage pipeline out-of-order 64- bit high performance RISC-v processor with vector extension : Industrial product," 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), pp. 52–64.

[5] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: https://www.tensorflow.org/

[6] Platzer, Michael, and Peter Puschner. "Vicuna: A Timing-Predictable RISC-V Vector Coprocessor for Scalable Parallel Computation", 2021. drops.dagstuhl.de, [Online]. Available: https://doi.org/10.4230/LIPIcs.ECRTS.2021.1.

[7] Francesco Minervini, Oscar Palomar, Osman Unsal, Enrico Reggiani, Josue Quiroga, Joan Marimon, Carlos Rojas, Roger Figueras, Abraham Ruiz, Alberto Gonzalez, Jonnatan Mendoza, Ivan Vargas, Cesar Hernandez, Joan Cabre, Lina Khoirunisya, Mustapha Bouhali, Julian Pavon, Francesc Moll, Mauro Olivieri, Mario Kovac, Mate Kovac, Leon Dragic, Mateo Valero, and Adrian Cristal, "Vitruvius+: An area-efficient RISC-v decoupled vector coprocessor for high performance computing applications".

[8] E. Casadesus, A. Valdivieso, M. Vargas and M. Montón, "Deploying Neural Networks on RISC-V with VPU," 2024 39th Conference on Design of Circuits and Integrated Systems (DCIS), Catania, Italy, 2024, pp. 1-6.

[9] Jones, J. P., van Kempen, P., and Müller-Gritschneder, D. (2025). Vicuna 2.0: A configurable RISC-V embedded vector hardware platform [Póster de conferencia]. RISC-V Summit Europe 2025, [Online]. Available: https://riscv-europe.org/summit/2025/media/proceedings/2025-05-13-RISC-V-Summit-Europe-P2.1.05-JONES-poster.pdf.

[10] Luffca. (2022). Running auto-vectorized program on RISC-V vector RTL simulator. https://www.luffca.com/2022/06/riscv-vector-vicuna-simulator/.

[11] OpenHW Group. OpenHW Group specification: Core-V eXtension interface (CV-X-IF) – Development. https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/latest/.

[12] vproc vicuna: RISC-V Zve32x vector coprocessor [Repositorio en GitHub]. GitHub. https://github.com/vproc/vicuna